

Pandas: Functions & Methods with Examples

A comprehensive practical reference for Data Science, AI/ML, EDA, data preprocessing, feature engineering, and analysis.

Recommended for: Python, Data Science, AI/ML and Machine Learning students

Table of Contents

1. Creating Pandas Objects
2. Reading and Writing Data
3. Inspecting Data
4. Selecting Rows and Columns
5. Filtering Data
6. Adding, Removing and Renaming Data
7. Missing Values
8. Duplicate Data
9. Sorting
10. Data Types
11. String Operations
12. Mathematical and Statistical Functions
13. `apply()`, `map()`, `replace()`, `where()`, `mask()`
14. GroupBy and Aggregation
15. Pivot, Melt and Crosstab
16. Combining DataFrames
17. Index Operations
18. Sampling and Ranking
19. Time Series and Datetime
20. Encoding and Binning
21. Advanced Transformations
22. Correlation and Covariance
23. Complete EDA Example
24. Top Pandas Functions for AI/ML

1. Creating Pandas Objects

pd.Series()

Creates a one-dimensional labeled data structure.

```
import pandas as pd data = pd.Series([10, 20, 30, 40]) print(data)
```

pd.DataFrame()

Creates a two-dimensional tabular data structure.

```
data = {"Name": ["Aftab", "Rahul", "Priya"], "Age": [21, 22, 20], "Marks": [90, 85, 88]} df = pd.DataFrame(data) print(df)
```

pd.date_range()

Creates a sequence of datetime values.

```
dates = pd.date_range(start="2026-01-01", periods=5, freq="D") print(dates)
```

2. Reading and Writing Data

pd.read_csv()

Reads a CSV file into a DataFrame.

```
df = pd.read_csv("students.csv") df = pd.read_csv("students.csv", usecols=["Name", "Age", "Marks"])
```

pd.read_excel()

Reads an Excel workbook.

```
df = pd.read_excel("students.xlsx") df = pd.read_excel("students.xlsx", sheet_name="Sheet1")
```

pd.read_json()

Reads JSON data.

```
df = pd.read_json("data.json")
```

pd.read_sql()

Reads the result of a SQL query into a DataFrame.

```
df = pd.read_sql("SELECT * FROM students", connection)
```

pd.read_html()

Extracts HTML tables.

```
tables = pd.read_html("https://example.com")
```

to_csv()

Exports a DataFrame to CSV.

```
df.to_csv("output.csv", index=False)
```

to_excel()

Exports a DataFrame to Excel.

```
df.to_excel("output.xlsx", index=False)
```

to_json()

Exports a DataFrame to JSON.

```
df.to_json("output.json")
```

to_sql()

Writes a DataFrame to a SQL table.

```
df.to_sql("students", connection, if_exists="replace", index=False)
```

3. Inspecting Data

head()

Shows the first rows.

```
df.head() df.head(2)
```

tail()

Shows the last rows.

```
df.tail() df.tail(3)
```

shape

Returns the number of rows and columns.

```
print(df.shape)
```

columns

Returns column labels.

```
print(df.columns)
```

index

Returns row index information.

```
print(df.index)
```

dtypes

Returns data types of columns.

```
print(df.dtypes)
```

info()

Displays concise DataFrame information, including non-null counts and dtypes.

```
df.info()
```

describe()

Generates descriptive statistics.

```
df.describe() df.describe(include="all")
```

memory_usage()

Reports memory usage.

```
df.memory_usage(deep=True)
```

unique()

Returns unique values in a Series.

```
df["Age"].unique()
```

nunique()

Counts unique values.

```
df["Age"].nunique()
```

value_counts()

Counts frequency of each value.

```
df["Name"].value_counts()
```

4. Selecting Rows and Columns

Column selection

Select one or multiple columns.

```
df["Name"] df[["Name", "Marks"]]
```

loc[]

Label-based row and column selection.

```
df.loc[0] df.loc[0:2, ["Name", "Marks"]]
```

iloc[]

Integer-position based selection.

```
df.iloc[0] df.iloc[0:2] df.iloc[0, 1]
```

5. Filtering Data

Boolean filtering

Filters rows using conditions.

```
df[df["Marks"] > 80] df[(df["Marks"] > 80) & (df["Age"] > 20)]
```

OR / NOT filtering

Use | for OR and ~ for NOT.

```
df[(df["Marks"] > 90) | (df["Age"] > 22)] df[~(df["Age"] > 21)]
```

query()

Provides readable expression-based filtering.

```
df.query("Marks > 80") df.query("Marks > 80 and Age > 20") minimum_marks = 80 df.query("Marks >= @minimum_marks")
```

isin()

Checks membership in a list of values.

```
df[df["Department"].isin(["IT", "HR"])]
```

between()

Checks whether values fall within an inclusive range.

```
df[df["Age"].between(18, 25)]
```

6. Adding, Removing and Renaming Data

Adding a column

Creates a new column.

```
df["Passed"] = df["Marks"] >= 40 df["Bonus"] = df["Marks"] * 0.10
```

drop()

Removes rows or columns.

```
df.drop(columns=["Age"]) df.drop(index=[0, 2])
```

rename()

Renames columns or index labels.

```
df.rename(columns={"Name": "Student_Name", "Marks": "Score"})
```

columns

Replace all column names at once.

```
df.columns = ["Student", "Age", "Score"]
```

insert()

Inserts a column at a specific position.

```
df.insert(1, "Country", "India")
```

pop()

Removes and returns a column.

```
marks = df.pop("Marks")
```

copy()

Creates an independent copy.

```
new_df = df.copy()
```

7. Missing Values

isna() / isnull()

Detects missing values.

```
df.isna() df.isna().sum() df.isnull().sum()
```

notna()

Detects non-missing values.

```
df["Age"].notna()
```

dropna()

Removes rows or columns containing missing values.

```
df.dropna() df.dropna(how="all")
```

fillna()

Replaces missing values.

```
df["Age"] = df["Age"].fillna(0) df["Age"] = df["Age"].fillna(df["Age"].median()) df["Name"] =  
df["Name"].fillna(df["Name"].mode()[0])
```

8. Duplicate Data

uplicated()

Identifies duplicate rows.

```
df.duplicated() df.duplicated().sum()
```

drop_duplicates()

Removes duplicate rows.

```
df = df.drop_duplicates() df = df.drop_duplicates(subset=["Email"], keep="first")
```

9. Sorting

sort_values()

Sorts by one or more columns.

```
df.sort_values("Marks") df.sort_values("Marks", ascending=False) df.sort_values(["Age", "Marks"], ascending=[True, False])
```

sort_index()

Sorts by index.

```
df.sort_index() df.sort_index(ascending=False)
```

nlargest()

Returns the largest n values.

```
df.nlargest(5, "Marks")
```

smallest()

Returns the smallest n values.

```
df.smallest(5, "Marks")
```

10. Data Types

astype()

Converts a column to a specified dtype.

```
df["Age"] = df["Age"].astype("int64") df["Age"] = df["Age"].astype("str")
```

pd.to_numeric()

Converts values to numeric, optionally coercing invalid values.

```
df["Marks"] = pd.to_numeric(df["Marks"], errors="coerce")
```

pd.to_datetime()

Converts values to datetime.

```
df["Date"] = pd.to_datetime(df["Date"])
```

pd.to_timedelta()

Converts values to timedelta.

```
df["Duration"] = pd.to_timedelta(df["Duration"])
```

11. String Operations

.str.lower()

Converts strings to lowercase.

```
df["Name"].str.lower()
```

.str.upper()

Converts strings to uppercase.

```
df["Name"].str.upper()
```

.str.title()

Converts strings to title case.

```
df["Name"].str.title()
```

.str.strip()

Removes leading and trailing whitespace.

```
df["Name"].str.strip()
```

.str.len()

Returns string lengths.

```
df["Name"].str.len()
```

.str.contains()

Tests whether a pattern occurs in strings.

```
df[df["Name"].str.contains("aftab", case=False, na=False)]
```

.str.startswith()

Tests string prefixes.

```
df[df["Name"].str.startswith("A")]
```

.str.endswith()

Tests string suffixes.

```
df[df["Name"].str.endswith("a")]
```

.str.replace()

Replaces text patterns.

```
df["Name"] = df["Name"].str.replace("Aftab", "Aftab Alam")
```

.str.split()

Splits strings; expand=True creates columns.

```
df[["First", "Last"]] = df["Name"].str.split(" ", expand=True)
```

12. Mathematical and Statistical Functions

sum()

Returns sum.

```
df["Marks"].sum()
```

mean()

Returns arithmetic mean.

```
df["Marks"].mean()
```

median()

Returns median.

```
df["Marks"].median()
```

mode()

Returns mode(s).

```
df["Marks"].mode()
```

min() / max()

Returns minimum and maximum.

```
df["Marks"].min() df["Marks"].max()
```

std()

Returns standard deviation.

```
df["Marks"].std()
```

var()

Returns variance.

```
df["Marks"].var()
```

quantile()

Returns quantiles.

```
df["Marks"].quantile(0.25) df["Marks"].quantile([0.25, 0.50, 0.75])
```

count()

Counts non-null values.

```
df["Marks"].count()
```

prod()

Returns product of values.

```
df["Marks"].prod()
```

abs()

Returns absolute values.

```
df["Difference"].abs()
```

round()

Rounds numerical values.

```
df["Salary"].round(2)
```

clip()

Limits values to lower and upper bounds.

```
df["Age"].clip(lower=18, upper=60)
```

13. apply(), map(), replace(), where(), mask()

apply()

Applies a function to Series or DataFrame values.

```
def grade(marks): if marks >= 90: return "A" elif marks >= 75: return "B" elif marks >= 60: return "C"
return "D" df["Grade"] = df["Marks"].apply(grade)
```

map()

Maps Series values using a dictionary or function.

```
df["Gender"] = df["Gender"].map({"M": "Male", "F": "Female"})
```

replace()

Replaces values or patterns.

```
df["Gender"] = df["Gender"].replace({"M": "Male", "F": "Female"})
```


where()

Keeps values where a condition is true and replaces others.

```
df["Marks"] = df["Marks"].where(df["Marks"] >= 40, 0)
```

mask()

Replaces values where a condition is true.

```
df["Marks"] = df["Marks"].mask(df["Marks"] < 40, 0)
```

14. GroupBy and Aggregation

groupby()

Groups rows by one or more keys.

```
df.groupby("Department")["Salary"].mean()
```

agg()

Performs multiple aggregations.

```
df.groupby("Department")["Salary"].agg(["mean", "sum", "min", "max"])
```

Named aggregation

Creates readable output column names.

```
df.groupby("Department").agg( average_salary=("Salary", "mean"), maximum_salary=("Salary", "max"),  
employee_count=("Salary", "count") )
```

15. Pivot, Melt and Crosstab

pivot_table()

Creates spreadsheet-style summaries and can aggregate duplicates.

```
pd.pivot_table( df, values="Salary", index="Department", columns="Gender", aggfunc="mean" )
```

pivot()

Reshapes without aggregation; duplicate index/column combinations cause an error.

```
df.pivot(index="Department", columns="Gender", values="Salary")
```

melt()

Converts wide-format data into long format.

```
pd.melt(df, id_vars=["Name"], var_name="Subject", value_name="Marks")
```

crosstab()

Builds frequency or aggregation tables.

```
pd.crosstab(df["Gender"], df["Department"]) pd.crosstab(df["Department"], df["Gender"],  
values=df["Salary"], aggfunc="mean")
```

16. Combining DataFrames

concat()

Concatenates DataFrames along rows or columns.

```
result = pd.concat([df1, df2], ignore_index=True) result = pd.concat([df1, df2], axis=1)
```

merge()

Performs SQL-style joins.

```
result = pd.merge(students, courses, on="student_id", how="inner") # how: inner, left, right, outer
```

join()

Joins using indexes.

```
result = df1.join(df2, lsuffix="_left", rsuffix="_right")
```

combine_first()

Uses values from another DataFrame to fill missing values.

```
result = df1.combine_first(df2)
```

update()

Updates values from another DataFrame.

```
df1.update(df2)
```

17. Index Operations

set_index()

Sets one or more columns as the index.

```
df = df.set_index("Name")
```

reset_index()

Moves the index back into columns.

```
df = df.reset_index()
```

reindex()

Reorders or creates labels according to a new index/column specification.

```
df = df.reindex(columns=["Name", "Marks", "Age"])
```

18. Sampling and Ranking

sample()

Returns random rows or a random fraction of rows.

```
df.sample(n=5) df.sample(frac=0.2, random_state=42)
```

rank()

Assigns ranks to values.

```
df["Rank"] = df["Marks"].rank(ascending=False, method="dense")
```

idxmax()

Returns the index of the maximum value.

```
index = df["Marks"].idxmax() top_student = df.loc[df["Marks"].idxmax()]
```

idxmin()

Returns the index of the minimum value.

```
index = df["Marks"].idxmin()
```

19. Time Series and Datetime

dt.year / dt.month / dt.day

Extracts calendar components.

```
df["Date"] = pd.to_datetime(df["Date"]) df["Year"] = df["Date"].dt.year df["Month"] = df["Date"].dt.month  
df["Day"] = df["Date"].dt.day
```

dt.month_name() / dt.day_name()

Extracts readable names.

```
df["Month_Name"] = df["Date"].dt.month_name() df["Day_Name"] = df["Date"].dt.day_name()
```

dt.dayofweek / dt.quarter

Extracts weekday number and quarter.

```
df["Day_Of_Week"] = df["Date"].dt.dayofweek df["Quarter"] = df["Date"].dt.quarter
```

shift()

Moves values by a specified number of periods.

```
df["Previous_Marks"] = df["Marks"].shift(1)
```

diff()

Computes differences between consecutive values.

```
df["Difference"] = df["Marks"].diff()
```

pct_change()

Computes percentage change.

```
df["Growth"] = df["Sales"].pct_change()
```

rolling()

Creates rolling windows for moving statistics.

```
df["Moving_Average"] = df["Sales"].rolling(window=3).mean()
```

expanding()

Calculates statistics over an expanding window.

```
df["Cumulative_Average"] = df["Sales"].expanding().mean()
```

resample()

Groups time-series data by a new time frequency.

```
df["Date"] = pd.to_datetime(df["Date"]) df = df.set_index("Date") monthly_sales =  
df["Sales"].resample("ME").sum()
```

cumsum() / cumprod() / cummax() / cummin()

Cumulative calculations.

```
df["Cumulative_Sales"] = df["Sales"].cumsum() df["Growth"] = df["Value"].cumprod() df["Maximum"] =  
df["Sales"].cummax() df["Minimum"] = df["Sales"].cummin()
```

20. Encoding and Binning

get_dummies()

Performs one-hot encoding.

```
encoded = pd.get_dummies(df, columns=["Gender"], dtype=int)
```

cut()

Creates bins using fixed numeric intervals.

```
df["Age_Group"] = pd.cut( df["Age"], bins=[0, 18, 30, 50, 100], labels=["Child", "Young", "Adult", "Senior"] )
```

qcut()

Creates quantile-based bins.

```
df["Marks_Group"] = pd.qcut( df["Marks"], q=4, labels=["Low", "Medium", "High", "Very High"] )
```

21. Advanced Transformations

assign()

Creates columns in a method chain without modifying the original DataFrame.

```
result = df.assign( Bonus=lambda x: x["Salary"] * 0.10, Total=lambda x: x["Salary"] + x["Bonus"] )
```

pipe()

Passes a DataFrame through custom functions.

```
def clean_data(dataframe): return dataframe.drop_duplicates().dropna() result = df.pipe(clean_data)
```

eval()

Evaluates expressions using column names.

```
df.eval("Total = Python + SQL") df.eval("Salary_After_Bonus = Salary * 1.10")
```

explode()

Turns list-like values into separate rows.

```
df = pd.DataFrame({ "Name": ["Aftab", "Rahul"], "Skills": [["Python", "SQL"], ["Java", "Python"]] }) result = df.explode("Skills")
```

transpose() / T

Transposes rows and columns.

```
df.transpose() df.T
```

any() / all()

Tests whether any or all values satisfy a condition.

```
df.isna().any() (df["Marks"] >= 40).all()
```

22. Correlation and Covariance

corr()

Computes pairwise correlation for numeric columns.

```
df.corr(numeric_only=True) df[["Age", "Salary", "Experience"]].corr()
```

cov()

Computes pairwise covariance.

```
df.cov(numeric_only=True)
```

equals()

Checks whether two DataFrames contain the same elements and structure.

```
df1.equals(df2)
```

23. Complete EDA Example

Basic EDA workflow

A practical starting point for a new dataset.

```
import pandas as pd
df = pd.read_csv("students.csv")
print("Shape:", df.shape)
print("\nColumns:")
print(df.columns)
print("\nData Types:")
print(df.dtypes)
print("\nInfo:")
df.info()
print("\nStatistics:")
print(df.describe())
print("\nMissing Values:")
print(df.isna().sum())
print("\nDuplicate Rows:")
print(df.duplicated().sum())
print("\nCorrelation:")
print(df.corr(numeric_only=True))
```

24. Top Pandas Functions for AI/ML

Priority learning list

Master these first for EDA, preprocessing and feature engineering.

```
1. read_csv() 2. DataFrame() 3. head() 4. info() 5. describe() 6. shape 7. loc[] 8. iloc[] 9. Boolean filtering
10. query() 11. isna() 12. fillna() 13. dropna() 14. drop_duplicates() 15. groupby() 16. agg() 17. merge() 18. concat() 19. apply() 20. get_dummies()
Then learn: pivot_table(), melt(), cut(), qcut(), rolling(), shift(), resample(), explode(), assign(), pipe()
```

Quick Reference: Pandas Workflow for AI/ML

```
1. Load data → pd.read_csv(), pd.read_excel() 2. Inspect → head(), shape, info(), describe() 3. Clean → isna(), fillna(), dropna(), drop_duplicates() 4. Select/filter → loc[], iloc[], query(), isin(), between() 5. Transform → apply(), map(), replace(), assign() 6. Explore → groupby(), agg(), pivot_table(), crosstab() 7. Combine → merge(), concat(), join() 8. Feature engineer → get_dummies(), cut(), qcut(), shift(), rolling() 9. Validate → corr(), describe(), value_counts() 10. Save → to_csv(), to_excel(), to_json(), to_sql()
```

Best practice: Prefer vectorized Pandas/NumPy operations over Python-level loops and row-by-row `apply()` when a vectorized alternative exists. Use method chaining and clear intermediate steps for maintainable data-processing pipelines.

Note: Pandas evolves over time. Some APIs and frequency aliases may change between versions; check the documentation for the exact version used in your project.